

Reinforcement Learning with Human Feedback (RLHF)

Vinay Setty

vinay.j.setty@uis.no



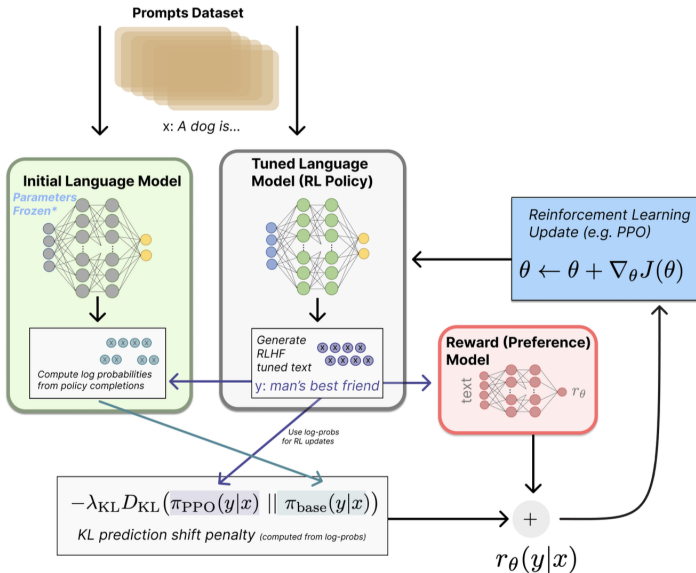
Department of Electrical Engineering and Computer Science
University of Stavanger

February 11, 2026



- ▶ Reward model $r_{\theta}(x, y)$ assigns a scalar score to a prompt-completion pair.
- ▶ Trained from pairwise preferences: chosen y_c vs. rejected y_r for the same prompt x .
- ▶ Used as proxy objective for policy optimization (e.g., PPO).

Reward Model in RLHF (Concept) (cont.)





Given two items i and j with positive latent strengths p_i, p_j :

$$P(i \succ j) = \frac{p_i}{p_i + p_j}.$$

Reparameterize $p_i = e^{r_i}$ with unbounded scores $r_i \in \mathbb{R}$:

$$P(i \succ j) = \frac{e^{r_i}}{e^{r_i} + e^{r_j}} = \sigma(r_i - r_j), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

- Only score differences matter: $r_i \leftarrow r_i + c$ leaves probabilities unchanged.



For a prompt x and two completions y_1, y_2 , the reward model produces:

$$r_{\theta}(y \mid x) \in \mathbb{R}.$$

Bradley–Terry probability that y_1 is preferred over y_2 :

$$P(y_1 \succ y_2 \mid x) = \frac{\exp(r_{\theta}(y_1 \mid x))}{\exp(r_{\theta}(y_1 \mid x)) + \exp(r_{\theta}(y_2 \mid x))}.$$

Let the chosen completion be y_c and rejected be y_r .

Pairwise RM Loss (Logistic / Logsigmoid Form)



Maximize log-likelihood of preference $y_c \succ y_r$:

$$\mathcal{L}(\theta) = -\log \sigma(r_\theta(y_c | x) - r_\theta(y_r | x)).$$

Equivalently, with $\Delta = r_\theta(y_c | x) - r_\theta(y_r | x)$:

$$\mathcal{L}(\theta) = -\log \sigma(\Delta).$$

Training a Preference RM: Pairwise Comparison at EOS

Chosen: <|eos|> Joy can ... ? The answer is 5 . <|eos|> Supervised

Rejected: <|eos|> Joy can ... ? The answer is 3 . <|eos|> Rejected

Loss: $\mathcal{L} = -\log \sigma(r_c - r_r)$ | Only score difference matters



Using the identity $-\log \sigma(\Delta) = \log(1 + e^{-\Delta})$:

$$\mathcal{L}(\theta) = \log \left(1 + \exp \left(r_{\theta}(y_r | x) - r_{\theta}(y_c | x) \right) \right).$$

- ▶ Same objective, different numerical style.
- ▶ Gradient depends only on the score difference.



If annotators provide a preference margin $m(y_c, y_r)$ (e.g., Likert scores):

$$\mathcal{L}(\theta) = -\log \sigma(r_\theta(y_c | x) - r_\theta(y_r | x) - m(y_c, y_r)).$$

- Encourages larger separation when the preference is stronger.

Outcome Reward Model (ORM)



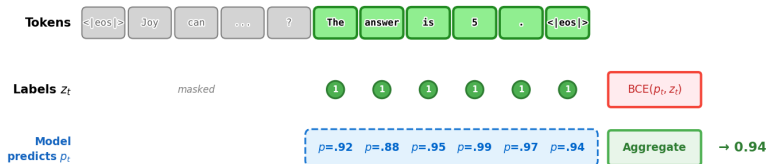
ORM predicts correctness probabilities per token in the completion.

Let $z_t \in \{0, 1\}$ be an outcome label applied on completion tokens (prompt tokens masked). With per-token logit $\ell_{\theta,t}$:

$$\mathcal{L}_{\text{ORM}}(\theta) = \sum_{t \in \text{completion}} \text{BCEWithLogits}(\ell_{\theta,t}, z_t).$$

Training Outcome RM: Offline Labels $\rightarrow$ Per-Token BCE

Labels from: Verifier / Dataset (offline)



Offline supervision • Fixed dataset labels • Use: verify / filter / rerank



PRM predicts step-level validity at reasoning step boundaries.

Let steps be $s_1, \dots, s_K$ with labels $y_{s_i} \in \{0, 1\}$ and predicted probability $r_\theta(s_i | x)$:

$$\mathcal{L}_{\text{PRM}}(\theta) = - \sum_{i=1}^K \left(y_{s_i} \log r_\theta(s_i | x) + (1 - y_{s_i}) \log(1 - r_\theta(s_i | x)) \right).$$





Some preferences are stronger than others, so we encode that in a margin.

For a chosen–rejected pair (y_c, y_r) with margin $m > 0$ and

$$\Delta = r_{\theta}(y_c \mid x) - r_{\theta}(y_r \mid x),$$

the margin-aware loss is

$$\mathcal{L}_{\text{margin}}(\theta) = -\log \sigma(\Delta - m) = \log \left(1 + e^{-(\Delta - m)} \right).$$

- ▶ Larger m forces a wider separation between scores.
- ▶ If $\Delta \geq m$, the loss is small.
- ▶ If $\Delta < m$, the model is penalized even if the ordering is correct.



Instead of comparing two completions, we compare k completions for the same prompt.

Let $\{y_1, \dots, y_k\}$ be ranked by human preference

$$y_1 \succ y_2 \succ \dots \succ y_k,$$

with rewards $r_i = r_\theta(y_i \mid x)$.

A common k-wise loss is the softmax cross-entropy:

$$\mathcal{L}_{k\text{-wise}}(\theta) = -\log \frac{\exp(r_1)}{\sum_{i=1}^k \exp(r_i)}.$$

- ▶ Reduces to pairwise loss when $k = 2$.
- ▶ Encourages correct ordering across all candidates at once.
- ▶ Uses more annotation signal per prompt but is harder to collect.



After training a reward model, we optimize the policy π_θ to maximize expected reward.

For a prompt x and completion $y \sim \pi_\theta(\cdot \mid x)$:

$$J(\theta) = \mathbb{E}[r(y \mid x)].$$

Policy-gradient methods update π_θ directly using samples from the current policy, rather than learning a separate action-value function.



Naive policy gradients are unstable because the policy can change too much in one update.

PPO constrains updates by comparing:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

The clipped surrogate objective:

$$\mathcal{L}_{\text{PPO}} = \mathbb{E} \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right].$$

- ▶ A_t is the advantage estimate.
- ▶ Clipping limits how much probabilities can change.

PPO in RLHF (Language Models)



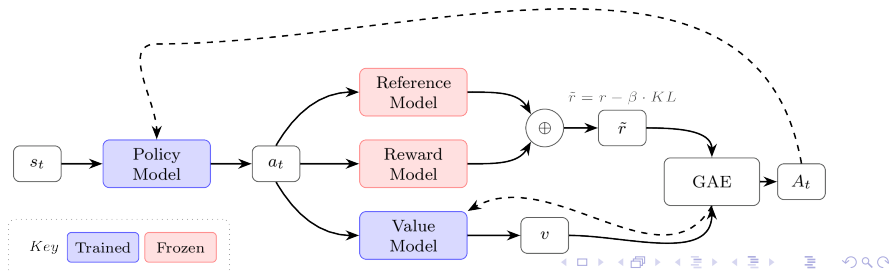
In RLHF, PPO is applied at the token level.

- Policy: the language model.
- Action: generating the next token.
- Reward: reward model score at EOS, plus KL penalty.

A learned value function $V_{\phi}(s_t)$ estimates future return, and advantages are computed via:

$$A_t = G_t - V_{\phi}(s_t),$$

often using Generalized Advantage Estimation (GAE).



Why PPO Works (and Hurts)



What PPO gives you:

- ▶ Stable updates via clipping.
- ▶ Token-level credit assignment.
- ▶ Well-studied behavior in Deep RL.

What PPO costs you:

- ▶ Extra value network.
- ▶ GAE complexity.
- ▶ High memory and tuning overhead.

This pain is exactly why GRPO exists.



GRPO removes the value function entirely.

For a prompt x , sample G completions:

$$y_1, \dots, y_G \sim \pi_\theta(\cdot \mid x),$$

with rewards $r_1, \dots, r_G$.

The advantage for completion i is computed relative to the group:

$$A_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}.$$

- ▶ No learned critic.
- ▶ Advantage is purely comparative.



GRPO uses a PPO-style clipped objective, but with group-relative advantages:

$$\mathcal{L}_{\text{GRPO}} = \mathbb{E} \left[\min \left(r_t(\theta) A_i, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_i \right) - \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}) \right].$$

Key differences from PPO:

- ▶ A_i is sequence-level, shared across tokens.
- ▶ KL penalty is added directly to the loss.



GRPO asks a simpler question than PPO:

“Which answers to the same prompt are better than the others?”

- ▶ If a completion beats the group average, increase its probability.
- ▶ If it is worse than average, suppress it.
- ▶ The scale comes from the group, not a value network.

This makes GRPO especially effective when:

- ▶ Multiple samples per prompt are cheap.
- ▶ Rewards are sparse or verifiable.

PPO vs. GRPO (Summary)



	PPO	GRPO
Value function	Yes	No
Advantage type	Token-level	Group-relative
Extra model	Yes (critic)	No
Memory cost	High	Lower
Best for	General RLHF	Reasoning, verifiable rewards



DPO is a Direct Alignment Algorithm that solves the RLHF objective without training a reward model or running policy gradient RL.

Core idea:

- ▶ Optimize the policy directly from preference pairs.
- ▶ Implicitly learn a reward via log-probability ratios.
- ▶ Enforce a KL constraint to stay close to a reference model.



DPO directly optimizes a policy from preference pairs, without training a reward model or running RL.

Starting from the KL-regularized RLHF objective:

$$\max_{\pi} \mathbb{E}[r(x, y)] - \beta D_{\text{KL}}(\pi(y | x) \| \pi_{\text{ref}}(y | x)).$$

The optimal policy has closed form:

$$\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right).$$

This implies an implicit reward:

$$r(x, y) \propto \beta \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}.$$

DPO replaces the reward model with this log-ratio.



Using a Bradley–Terry preference model:

$$p(y_c \succ y_r \mid x) = \sigma \left(\beta \log \frac{\pi(y_c \mid x)}{\pi_{\text{ref}}(y_c \mid x)} - \beta \log \frac{\pi(y_r \mid x)}{\pi_{\text{ref}}(y_r \mid x)} \right).$$

The DPO loss is:

$$\mathcal{L}_{\text{DPO}} = -\log p(y_c \succ y_r \mid x).$$

Effect:

- ▶ Increase likelihood of chosen response.
- ▶ Decrease likelihood of rejected response.
- ▶ Stay close to the reference via β .

